GLM-5 Cloud vs Qwen 3.5 vs Gemma 4 - Comprehensive Comparison

Test Date: April 8, 2026
Models Compared:
- `qwen3.5:397b-cloud` (OpenClaw current standard)
- `gemma4:31b-cloud` (Google's latest)
- `glm-5:cloud` (NEW - Zhipu's flagship)

Executive Summary

■ The Verdict: GLM-5 cloud is **powerful but SLOW** for coding agent tasks compared to Qwen 3.5 and Gemma 4.

At a Glance

Metric	Winner	Notes
Speed/Latency	■ Gemma 4	19.7s code generation vs GLM-5's 72.9s
Code Quality	■ Qwen 3.5	Most concise, production-ready
Comprehensive Analysis	■ GLM-5	Longest, most detailed responses
Overall Coding	■ Qwen 3.5	Best balance of speed + quality
Multi-step Reasoning	■ Qwen 3.5	Faster on complex refactoring
Detailed Reviews	■ GLM-5	Most thorough security analysis

Performance Metrics: Speed Comparison

Total Testing Time
- **Qwen 3.5:** 256.4 seconds (4.27 min)
- **Gemma 4:** 439.9 seconds (7.33 min)
- **GLM-5:** 808.9 seconds (13.48 min) ■■

GLM-5 is 3.1x slower than Qwen 3.5 overall

Per-Test Breakdown

Test	Qwen 3.5	Gemma 4	GLM-5	Fastest	Slowest
1. Code Generation	75.1s	19.7s ■	72.9s	Gemma (4x faster)	Qwen/GLM similar
2. Bug Fixing	13.9s ■	84.0s	171.7s	Qwen (12.3x faster)	GLM (12.3x slower)
3. Code Review	19.0s ■	48.8s	104.2s	Qwen (5.5x faster)	GLM (5.5x slower)
4. Refactoring	28.2s ■	25.1s	92.0s	Gemma (3.7x faster)	GLM (3.7x slower)
5. Test Writing	49.8s ■	101.0s	167.2s	Qwen (3.4x faster)	GLM (3.4x slower)
6. Architecture	70.4s ■	78.3s	201.2s	Qwen (2.9x faster)	GLM (2.9x slower)

Key Finding
GLM-5 is consistently 2-12x slower, with the worst performance on bug fixing (171.7s vs Qwen's 13.9s) and architecture design (201.2s vs Qwen's 70.4s).

Code Quality Analysis

Test 1: Code Generation - `merge_sorted_streams()`

Word Count:
- Qwen 3.5: 132 words
- Gemma 4: 172 words
- GLM-5: 209 words ← Most verbose

Quality Assessment:

Model	Approach	Quality	Notes

Qwen 3.5	Compact & direct	■■■■■	Perfect - minimal, working code
Gemma 4	Moderate verbosity	■■■■	Good, includes comments
GLM-5	Verbose with extras	■■■■■	Most robust - adds `from __future__` imports, extensive comments

Winner: All three produce working code. Qwen is most concise; GLM-5 is most defensive with Python 3.10+ annotations.

Test 2: Bug Fixing - ThreadSafe RateLimiter

Analysis Quality:
- **Qwen 3.5 (13.9s):** Identifies bugs quickly, provides fixes
- **Gemma 4 (84.0s):** More detailed explanations
- **GLM-5 (171.7s):** EXTREMELY detailed, but **takes 12x longer**

Key Bug Identification:
All three correctly identified:
- ■ Missing lock in `get_remaining()` method
- ■ Race condition in `reset()` method
- ■ Potential `KeyError` in `get_wait_time()`

GLM-5 Advantage: Added extra edge case handling for empty client IDs and better thread-safety patterns.

Speed Issue: GLM-5's verbose approach makes it impractical for quick bug fixes in production scenarios.

Test 3: Code Review - Vulnerable REST API

Response Lengths:
- Qwen 3.5: 780 words (19.0s)
- Gemma 4: 752 words (48.8s)
- GLM-5: 1,105 words (104.2s) ← Most comprehensive

Security Issues Found (all three models):
1. ■ **SQL Injection in search endpoint** - Identified by all
2. ■ **SQL Injection in delete endpoint** - Identified by all
3. ■ **Plaintext password storage** - Identified by all
4. ■ **Missing authentication/authorization** - Identified by all
5. ■ **Sensitive data exposure** - Identified by all (Qwen added SELECT * issue)

Quality Differences:

Model	Format	Severity Labels	Code Examples	Recommendations
Qwen 3.5	Markdown with emojis	■ Critical, ■ High	Shows vulnerable code	Clear fixes
Gemma 4	Clean markdown	Listed clearly	Basic examples	Detailed mitigations
GLM-5	Detailed prose	"Critical", "High"	Rich context	Extremely thorough

Winner for Audits: GLM-5 provides the most comprehensive analysis, but Qwen's emoji-based formatting is more scannable for quick reviews.

Test 4: Refactoring - Nested Data Processing

Output Quality:

Model	Clarity	Explanation	Time
Qwen 3.5	■■■■■	Clear decisions	28.2s ■
Gemma 4	■■■■	Good explanations	25.1s ■
GLM-5	■■■■	Thorough notes	92.0s ■■

Winner: Gemma 4 (fastest with good quality)

Test 5: Test Writing - LRU Cache Jest Tests

Output Metrics:
- Qwen 3.5: 1,039 words (49.8s)
- Gemma 4: 750 words (101.0s)
- GLM-5: 984 words (167.2s)

Test Coverage Comparison:

Aspect	Qwen 3.5	Gemma 4	GLM-5
Basic operations	■	■	■
Edge cases	■	■	■
Concurrent access	Partial	■	■ Rich
DX/Readability	■ Best	■	Verbose

GLM-5 Strength: Most thorough test cases with detailed comments, but takes 3.4x longer.

Test 6: Architecture - Webhook Delivery System

Output Sizes:
- Qwen 3.5: 1,960 words (70.4s)
- Gemma 4: 814 words (78.3s)
- GLM-5: 1,589 words (201.2s) ← Slowest

Architecture Quality:

Component	Qwen 3.5	Gemma 4	GLM-5
Database schema	■ Complete	■ Good	■ Detailed
Delivery service	■ TypeScript	■ TS	■ TS + error handling
Reliability patterns	■ DLQ + retries	■ Basic	■ Comprehensive
Implementation details	■ Production-ready	Medium	■ Very thorough

Winner: Qwen 3.5 - best value (detailed + fast)

Pros & Cons Summary

■ Qwen 3.5:397b-cloud

Strengths:
- ■ **FASTEST** for most coding tasks (2.9-12x faster than GLM-5)
- ■ High-quality, production-ready code
- ■ Perfect balance of speed and quality
- ■ Excellent reasoning for refactoring and bug fixing
- ■ Concise responses (easy to scan)

Weaknesses:
- Sometimes less verbose on architectural decisions
- Code generation responses slightly less detailed than GLM-5

Best For:
- Fast iteration loops
- Production code generation
- Time-sensitive bug fixes
- Real-time agent tasks (OpenClaw)

Overall Rating: ■■■■■ (9.5/10)

■ Gemma 4:31b-cloud

Strengths:
- ■ Fastest at code generation (19.7s)
- ■ Good balance between verbosity and speed
- ■ Decent refactoring output (25.1s)
- ■ Competent code review capabilities

Weaknesses:
- ■ Slower at bug fixing (84.0s) and test writing (101.0s)
- Architecture design takes longer than Qwen

Best For:
- When you need the fastest code generation

- Lightweight refactoring tasks
- Medium-complexity reviews

Overall Rating: ■■■■ (7.5/10)

■ GLM-5:cloud (NEW - Zhipu's Flagship)

Strengths:
- ■ **MOST COMPREHENSIVE** analysis and responses
- ■ Best for deep security reviews (1,105 words on API review)
- ■ Detailed architecture design with rich explanations
- ■■ Defensive coding practices (extra error handling)
- ■ Excellent at multi-step complex reasoning

Weaknesses:
- ■ **3x SLOWER** than Qwen 3.5 overall
- ■■ **12x slower** on bug fixing (171.7s vs 13.9s)
- ■ Responses are too verbose for real-time agent tasks
- Not suitable for latency-sensitive applications
- Not practical for rapid iteration

Best For:
- One-time comprehensive code reviews
- Architectural design documents
- Security-focused analysis
- Non-time-critical documentation
- Academic or detailed technical analysis

Best When: You have time for detailed responses (not in production chat loop)

Overall Rating: ■■■ (6.5/10) - Great for depth, but too slow for most use cases

Use Case Recommendations

For OpenClaw Agent Framework

Scenario	Recommended Model	Reason
Primary coding agent	■ Qwen 3.5	Fast + quality = best for agents
Quick bug fixes	■ Qwen 3.5	13.9s vs GLM-5's 171.7s
Code generation	■ Gemma 4	Fastest at 19.7s for initial scaffolding
Deep security review	■ GLM-5	Use offline/batch mode, not real-time
Fallback (fastest)	■ Gemma 4	When speed matters most
Complex architecture	■ Qwen 3.5	Good speed (70.4s) + comprehensive

Recommendation

■ VERDICT: Keep Qwen 3.5 as primary; skip GLM-5 for production agents

Why?
1. **Speed matters for agents** - User perception of 3x slower responses ruins UX
2. **Qwen 3.5 already excellent** - No quality advantage to justify 3x latency
3. **Use GLM-5 for batch analysis** - Run it offline for deep audit reports
4. **Gemma 4 still useful** - Fastest code generation (keeps as backup)

Suggested Configuration

```json
{
"agents": {
"coding": {
"primary": "ollama/qwen3.5:397b-cloud",
"fallback": "ollama/gemma4:31b-cloud",
```

"batch_analysis": "ollama/glm-5:cloud"
}
}
}
```

### When to Use GLM-5:cloud

■ **DO USE:**
- Nightly security audit reports
- Archive/batch code review processes
- Detailed architectural documentation
- Complex system design documents
- Compliance/regulatory analysis

■ **DON'T USE:**
- Real-time agent tasks
- Interactive coding sessions
- Time-sensitive bug fixes
- Production code generation loops
- User-facing features

## Test Files

All detailed results are saved in `./test-results/`:

**Qwen 3.5 Results:**
- `qwen3.5_397b-cloud_1_code_generation.md` (132 words, 75.1s)
- `qwen3.5_397b-cloud_2_bug_fixing.md` (357 words, 13.9s)
- `qwen3.5_397b-cloud_3_code_review.md` (780 words, 19.0s)
- `qwen3.5_397b-cloud_4_refactoring.md` (713 words, 28.2s)
- `qwen3.5_397b-cloud_5_test_writing.md` (1,039 words, 49.8s)
- `qwen3.5_397b-cloud_6_architecture.md` (1,960 words, 70.4s)

**Gemma 4 Results:**
- `gemma4_31b-cloud_1_code_generation.md` (172 words, 19.7s)
- `gemma4_31b-cloud_2_bug_fixing.md` (433 words, 84.0s)
- `gemma4_31b-cloud_3_code_review.md` (752 words, 48.8s)
- `gemma4_31b-cloud_4_refactoring.md` (583 words, 25.1s)
- `gemma4_31b-cloud_5_test_writing.md` (750 words, 101.0s)
- `gemma4_31b-cloud_6_architecture.md` (814 words, 78.3s)

**GLM-5 Results:**
- `glm-5_cloud_1_code_generation.md` (209 words, 72.9s)
- `glm-5_cloud_2_bug_fixing.md` (453 words, 171.7s)
- `glm-5_cloud_3_code_review.md` (1,105 words, 104.2s)
- `glm-5_cloud_4_refactoring.md` (876 words, 92.0s)
- `glm-5_cloud_5_test_writing.md` (984 words, 167.2s)
- `glm-5_cloud_6_architecture.md` (1,589 words, 201.2s)

## Conclusion

GLM-5 is impressive for **depth and comprehensiveness** but **too slow for production agent use**.

Keep **Qwen 3.5** as your primary model—it delivers the best value for OpenClaw's agentic workflow. GLM-5 excels for non-urgent, batch-mode analysis where thoroughness beats speed.

**The future of Ollama's coding models is bright—use the right tool for the right job!** ■
```